Exploring Secure Coding Practices

Part A:

Case Study 1: SQL Injection in an Online Examination System

Scenario

A university develops an online examination management system. The application allows teachers to search student records using a student ID or name. The search functionality constructs database queries dynamically from user-provided input.

During security testing, the following behavior is observed:

- The application accepts database-related characters in the search field.
- User input is directly incorporated into SQL statements.
- Database errors are displayed to users.
- Different responses are observed when specially crafted input is entered.
- The application uses a database account with more privileges than required.
- The development team has not implemented centralized input validation.

Security Concerns

The development team needs to investigate:

1. Whether user input is being treated as data rather than executable SQL.
2. Whether parameterized queries or prepared statements are being used.
3. Whether database errors expose internal information.
4. Whether the application's database account follows the principle of least privilege.
5. Whether input validation is implemented consistently.
6. Whether security testing is performed before deployment.

Tasks

1. Identify the secure coding weaknesses in the application.
2. Explain how parameterized queries can prevent SQL injection.
3. Identify appropriate input-validation strategies.
4. Explain why database error messages should not be displayed directly to users.
5. Suggest appropriate database privileges for the application.
6. Design a secure coding checklist for database-related functionality.
7. Explain how code review and security testing could identify these problems before deployment.

Case Study 2: Cross-Site Scripting in an E-Commerce Website

Scenario

An e-commerce company develops a product-review feature. Customers can submit reviews that are displayed on the product page.

After deployment, security testers notice that:

- Review content is displayed without appropriate output encoding.
- The application accepts HTML-like input from users.
- Review data is stored in the database and displayed to other customers.
- The application does not consistently validate input.
- Some pages use client-side rendering to insert user-controlled content into the DOM.
- Session cookies do not consistently use appropriate security attributes.

The development team initially considers removing certain characters such as `<` and `>` from input. However, testers point out that relying only on character filtering may not adequately address all contexts in which untrusted data is displayed.

Security Concerns

The team must examine:

- Input validation
- Context-aware output encoding
- Safe DOM manipulation
- Content Security Policy
- Cookie security
- Secure handling of user-generated content

Tasks

1. Identify the potential types of XSS involved.
2. Explain the difference between input validation and output encoding.
3. Explain why context-aware encoding is important.
4. Identify safer approaches for inserting untrusted content into web pages.
5. Explain how Content Security Policy can provide an additional security layer.
6. Identify appropriate security attributes for session cookies.
7. Develop a secure coding checklist for user-generated content.
8. Explain how automated and manual security testing could be incorporated into the development lifecycle.

Case Study 3: Insecure Authentication and File Upload Functionality

Scenario

A hospital develops an internal document-management application. Authorized employees can log in and upload medical documents.

During a security assessment, the following issues are discovered:

- Passwords are stored using an outdated hashing approach.
- There is no effective protection against repeated login attempts.
- Password-reset links remain valid for an unnecessarily long period.
- Uploaded files are stored directly in a publicly accessible directory.
- The application determines the permitted file type primarily from the filename extension.
- Uploaded files are not subjected to adequate size and content validation.
- Authorization checks are inconsistently implemented.
- Sensitive information is included in application logs.

Security Concerns

The development team must consider:

- Secure password storage
- Authentication controls
- Session management
- Password-reset security
- File-upload validation
- Authorization
- Secure logging
- Protection of sensitive information

Tasks

1. Identify the secure coding weaknesses.
2. Explain how passwords should be securely stored.
3. Suggest controls for repeated failed authentication attempts.
4. Identify characteristics of a secure password-reset mechanism.
5. Design a secure file-upload validation strategy.
6. Explain why uploaded files should not automatically be stored in a publicly accessible location.
7. Distinguish authentication from authorization.
8. Identify sensitive information that should not be unnecessarily written to logs.
9. Develop a secure coding checklist for the application.
10. Explain how secure code review, dependency scanning, and penetration testing could complement each other.

Part B: 25 MSQs

Multiple Select Questions — More Than One Option May Be Correct

1. Which practices help prevent SQL injection?

A. Using parameterized queries
B. Using prepared statements
C. Concatenating user input directly into SQL
D. Applying appropriate input validation
E. Giving the application unrestricted database privileges

2. Which are characteristics of secure input validation?

A. Validation is performed on security-sensitive inputs
B. Validation considers the expected data type
C. Validation is based only on client-side JavaScript
D. Unexpected input is rejected or safely handled
E. Validation rules are defined according to the application's context

3. Which practices help reduce XSS vulnerabilities?

A. Context-aware output encoding
B. Safe DOM APIs
C. Content Security Policy
D. Displaying untrusted HTML without sanitization
E. Appropriate input validation

4. Which practices are associated with secure password storage?

A. Password hashing using an appropriate password-hashing algorithm
B. Using a unique salt for each password
C. Storing plaintext passwords
D. Selecting appropriate work factors
E. Encrypting all passwords with one common key and treating them as plaintext-equivalent data

5. Which properties are desirable in secure authentication mechanisms?

A. Protection against excessive failed login attempts
B. Secure session management
C. Secure password-reset mechanisms
D. Returning detailed authentication failure information that reveals which accounts exist
E. Appropriate multi-factor authentication where required

6. Which practices support secure session management?

A. Securely generating session identifiers
B. Regenerating session identifiers at appropriate privilege changes
C. Protecting session cookies with appropriate attributes
D. Placing sensitive session identifiers in URLs unnecessarily
E. Expiring sessions according to security requirements

7. Which cookie attributes can contribute to web application security?

A. Secure
B. HttpOnly
C. SameSite
D. Public
E. Path, when appropriately configured

8. Which are appropriate secure file-upload practices?

A. Restricting permitted file types
B. Applying file-size limits
C. Validating file content where appropriate
D. Storing uploaded files in a controlled location
E. Trusting only the filename extension as proof of file type

9. Why is server-side validation important?

A. Client-side controls can potentially be bypassed
B. The server is responsible for enforcing security-sensitive rules
C. It provides a trusted enforcement point

D. JavaScript validation eliminates the need for server-side validation
E. It helps ensure consistent security checks

10. Which practices follow the principle of least privilege?

A. Giving users only required permissions
B. Giving database accounts only required privileges
C. Giving every application administrative privileges
D. Restricting access to sensitive resources
E. Reviewing privileges periodically

11. Which practices help prevent command injection?

A. Avoiding unnecessary execution of operating-system commands
B. Using safe APIs instead of constructing shell commands where possible
C. Validating and constraining user-controlled arguments
D. Passing unrestricted user input directly to a shell
E. Applying least privilege to the executing process

12. Which practices contribute to secure error handling?

A. Logging appropriate technical information securely
B. Showing users generic error messages when detailed information could expose internals
C. Exposing complete database errors to end users
D. Separating user-facing errors from internal diagnostic information
E. Protecting application logs from unauthorized access

13. Which statements about secure logging are correct?

A. Logs should avoid unnecessary sensitive information
B. Logs should be protected against unauthorized modification
C. Security-relevant events should be recorded appropriately
D. Passwords should be recorded to simplify troubleshooting
E. Logs should have appropriate access controls

14. Which practices help prevent path traversal vulnerabilities?

A. Validating user-controlled file paths
B. Restricting access to permitted directories
C. Canonicalizing paths where appropriate
D. Directly trusting `../`-containing user input
E. Avoiding unnecessary exposure of filesystem paths

15. Which are useful secure coding practices for authorization?

A. Checking authorization on the server
B. Verifying that the current user can access the requested resource
C. Relying solely on hidden UI buttons
D. Applying access control consistently across relevant endpoints
E. Testing unauthorized and cross-user access attempts

16. Which practices can reduce Cross-Site Request Forgery (CSRF) risks?

A. Using appropriate anti-CSRF mechanisms
B. Correctly configuring SameSite cookies where suitable
C. Requiring appropriate authentication for sensitive operations
D. Assuming that authentication alone always prevents CSRF
E. Validating requests for sensitive state-changing operations

17. Which practices support secure API development?

A. Strong authentication and authorization
B. Input validation
C. Rate limiting where appropriate
D. Returning unnecessary sensitive information
E. Consistent access control across API endpoints

18. Which practices help secure dependencies?

A. Maintaining an inventory of dependencies
B. Monitoring known vulnerabilities
C. Applying security updates appropriately

D. Blindly copying outdated third-party code
E. Reviewing dependencies before introducing them into a project

19. Which activities are useful during secure code review?

A. Identifying unsafe input handling
B. Reviewing authentication and authorization logic
C. Checking sensitive-data handling
D. Looking for insecure API usage
E. Checking only whether the program produces the expected output

20. Which practices are appropriate for handling sensitive information?

A. Minimize collection and storage
B. Protect sensitive data during transmission
C. Apply appropriate access controls
D. Store sensitive information indefinitely without justification
E. Avoid exposing sensitive data through logs and error messages

21. Which practices can help prevent insecure deserialization?

A. Avoiding unnecessary deserialization of untrusted data
B. Validating data before processing
C. Using safer data formats where appropriate
D. Deserializing arbitrary untrusted objects without restrictions
E. Applying integrity and authenticity controls where appropriate

22. Which practices support secure software development throughout the SDLC?

A. Threat modeling
B. Secure coding standards
C. Security-focused code review
D. Security testing before release
E. Waiting until production to consider security

23. Which statements about hard-coded secrets are correct?

A. API keys should not normally be embedded directly in source code
B. Secrets should be managed through appropriate secret-management mechanisms
C. Secrets committed to source repositories may be exposed
D. Obscuring a password through simple encoding makes it secure
E. Secret rotation can reduce the impact of compromised credentials

24. Which practices can help reduce race-condition vulnerabilities?

A. Identifying operations that require atomicity
B. Applying appropriate synchronization mechanisms
C. Carefully controlling concurrent access to shared resources
D. Assuming sequential execution in a concurrent environment
E. Testing security-sensitive operations under concurrent conditions

25. Which activities provide useful layers of secure coding assurance?

A. Static application security testing
B. Dynamic application security testing
C. Manual code review
D. Dependency and software composition analysis
E. Relying exclusively on one automated security scanner